

Inflation

EGOI 2023

Problem author: Pak Hei Chan.

The first task of Day 1 of EGOI 2023 is relatively straightforward, utilizing commonly used techniques that are widely recognized. Hopefully, completing this task will provide advantageous assistance for you in achieving your dream of earning a medal in EGOI 2023.

Subtasks may not be ordered for ease of discussion.

1 The Problem

Given an array $p_1, p_2, \dots, p_N$, support the following two types of operations (there are Q operations in total):

- **INFLATION x**: Add p_i by x , for all $1 \leq i \leq N$.
- **SET x y**: For all i ($1 \leq i \leq N$) such that $p_i = x$, set $p_i = y$.

After each operation output the sum of the array.

2 Subtask 2: Small N and Q

Here, $N, Q \leq 100$. To solve this subtask, one can simulate the process described in the problem statement. Each of the two types of operations can be handled in $O(N)$ time, by going through the array and changing the values appropriately. Therefore, the overall complexity is $O(NQ)$.

Since $N, Q, p_i, x, y \leq 100$, C++ contestants do not have to worry about integer overflow problems.

Expected score: 28.

3 Subtask 1: $N = 1$

In this subtask, a $O(NQ)$ solution works as well. Here difference from subtask 2 is that contestants are not required to have knowledge about arrays. Also, C++ contestants have to be careful about integer overflow problems in this subtask (and all future subtasks): one need to use 64-bit integers (long long) to solve this subtask.

Expected score: 14 for just subtask 1, $28 + 14 = 42$ for subtask 1 and 2.

4 Subtask 3: Only INFLATION

In this subtask there are only **INFLATION** events. Notice that an event of the form **INFLATION** x adds the sum of the array by $N \cdot x$. Therefore, one can simply compute the sum of the array at first, then for every event **INFLATION** x , add the sum of the array by $N \cdot x$. There is no need to add each element of the array by x , and compute the sum of the array again.

Overall complexity: $O(N + Q)$.

Expected score: 19.

5 Subtask 4: Only SET

In this subtask there are only **SET** events. The key idea here is to try make our computations constant time. Notice that the indices in the array do not matter, what matters is the values.

Therefore, we will maintain a frequency array $f_1, f_2, \dots, f_{10^6}$, where f_i counts the number of items in the array p with value i .

Let the sum of the array before an operation be s . After a **SET** x y event, the following happens:

- $s := s + f_x \cdot (y - x)$, update the sum of the array.
- $f_y := f_y + f_x$, all indices that have value x now have value y instead.
- $f_x := 0$, there are no more indices that have value x now.

There is a **special case** where $x = y$. In this case nothing happens. If one simulates the above process it will not work. Careful handling is required to pass this subtask.

Overall complexity: $O(N + Q + C)$, where $C = 10^6$ is the maximum value in the array.

Expected score: 23.

6 Full Task

To get the final 16 points, we merge the ideas of subtask 3 and subtask 4.

Consider maintaining a `global` tag that keeps track of the total value added to each element of the array. Initially, `global` = 0.

That means, for each element, if its initial value is p_i , its current value must be equal to $p_i + \text{global}$.

This idea is important for understanding the following parts of the solution.

From subtask 4, we will maintain two things:

- Sum of the array s , ignoring global updates.
- Frequency array f . However, here the values can exceed the range $[1, 10^6]$. Hence, we should use a `unordered_map` (in C++) or dictionary (in Python).

For an `INFLATION x` operation:

- Add the `global` tag by x . There is no need to update s .

For an `SET x y` operation:

- We want to set all elements that have value x to have value y instead.
- For an element that currently has value x , its initial value must have been $x - \text{global}$.
- These elements will then have a current value of y , which corresponds to an initial value of $y - \text{global}$.
 - Look back at the key idea above if it doesn't make sense. If an element has an initial value of $y - \text{global}$, its current value equals y , which matches the objective of the operation.
- Based on the above two points, we subtract the inputs x and y by `global`.
- Perform the steps from subtask 4, if $x \neq y$:
 - $s := s + f_x \cdot (y - x)$
 - $f_y := f_y + f_x$
 - $f_x := 0$

Finally, the output after each operation is $s + N \cdot \text{global}$, since s does not take into account of global updates.

Time complexity: $O(N + Q)$, or $O(N + Q \log(N + Q))$ if a `map` is used instead of `unordered_map` in C++.

Expected score: $28 + 14 + 19 + 23 + 16 = 100$.

7 Hard Version

The originally proposed version of this problem was more difficult (but solvable). For those who are interested, you may try thinking of the solution.

Given an array $p_1, p_2, \dots, p_N$, support the following **four** types of operations/queries (there are Q operations in total):

1. Given i, x , do $p_i := p_i + x$.
2. Given x , do $p_i := p_i + x$ for all $1 \leq i \leq N$. (INFLATION)
3. Given i , output p_i .
4. Given x and y , for all i ($1 \leq i \leq N$) such that $p_i = x$, set $p_i = y$. (SET)

8 Solution to Hard Version

Consider grouping the initial array elements into equivalence classes, such that in each equivalence class, all elements have equal value. The idea is to use four maps/arrays:

1. Value $\rightarrow$ equivalence class
2. Equivalence class $\rightarrow$ value
3. Equivalence class $\rightarrow$ set of indices
4. Index in array $\rightarrow$ equivalence class

By maintaining these four maps/arrays for every type of operation, it can be easily seen that all types of operations can be solved. The tricky part is in the SET operations, the essential thing that has to be done is to merge two equivalence classes. We can naively insert all elements from the smaller set to the larger set, then clear the smaller set. This technique is known as **small-to-large merging**, and it ensures the total number of insert operations is $O((N + Q) \log N)$.

Padel Prize Pursuit

EGOI 2023

Problem authors: Pavle Martinović and Mladen Puzić

1 Two players – $N = 2$

For each medal we can determine who won more matches after that day and thus receives this medal in the end. If we do this starting with the last match, we can do this for all medals in $O(M)$ so the total complexity is $O(N + M)$.

2 Simulation – $N, M \leq 2000$

We can simulate the process by going through matches and maintaining a list of medals for each participant. For each pair of participant and medal we keep the number of nights that that participant held that medal. Then for each medal, we determine who held it the longest. Complexity: $O((N + M)M)$.

3 The winner of the i th match participates in the $(i + 1)$ th match for all i

For each medal, we need to determine the participant with the most wins. Observe that as the winner always plays the next match, all medals will always stay together. Like in Subtask 1, we can now go through the days starting at the last day, maintaining the number of wins and keeping track of which participant has the most wins. Complexity: $O(N + M)$.

4 At every match, the winner has at least as many medals as the loser

From the fact that the match is always won by the participant with the larger amount of medals at the moment, we can deduce that each medal will switch hands at most $O(\log M)$ times, as if we look at a fixed medal, the number of medals the person who holds this medal has doubles for each change of hands. Thus, we can simulate the process, but this time, in contrast to Subtask 2, we

can not keep days held for each medal and participant. However, there will be at most $O(\log M)$ medal holders per medal, so we can just remember all of them and find who held it the most nights in the end. Complexity: $O(M \log M + N)$.

5 Once a participant loses, they are never in a match again

Construct a tree where each match is a node. The parent of node x is the next match where the winner of match x appears. Add a fake root, so we turn the forest into a tree. The lifetime of each medal is a path from its first match to the root. All matches of a participant are consecutive on the path from the match where she first appeared to the root. This means that we can do DFS on the tree, starting from the root, and maintain who held each medal the longest from the root to the current node and also the current medal holder. Complexity $O(N + M)$.

6 Full Score

Construct the same tree as in the Subtask 5. Participants' matches are no longer a single path in the tree. Instead of keeping just the current and the longest medal holder, we need to keep the count for every participant and also maintain the longest medal holder. This can be done by keeping track of how many medals on a path a participant has one and reversing the changes when going up in the DFS tree. Complexity $O(N + M)$.

Find the Box

EGOI 2023

Problem author: Nils Gustafsson.

The Problem

There is an $H \times W$ grid. In some unknown cell there is a box. Your goal is to find the box.

Every night, a robot starts in the top left corner, and moves around the grid. You can decide how the robot should move by giving it instructions in the form of a string consisting of characters $<$, $>$, $\wedge$, $\vee$.

The walls are solid, so if the robot attempts to move outside of the grid, nothing will happen. The box is also solid, so you cannot move into the cell containing the box.

At the end of each night, the robot will report to you its location, and go back to the top left corner. Your task is to find the location of the box in at most Q nights. In particular, you get full score if $Q \leq 2$.

Solution 1: $Q = 50$ (20 points)

Notice that if the robot is on $(r, 0)$, and the box is on (r, c) , where $c > 0$, you may instruct the robot to keep going right and it will eventually get stuck on $(r, c - 1)$.

On the other hand, if the box is not on row r , the robot reaches $(r, W - 1)$.

Therefore, you can brute force each row. For row r , move r times downwards, then move $W - 1$ times rightwards and see if the box is stuck.

But be careful of the special case when the box is in column 0.

When $H = 50$, this solution uses $H = 50$ queries, which scores 20 points.

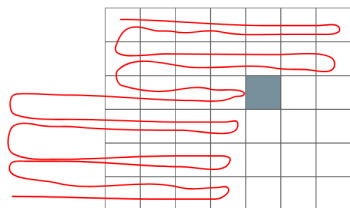
Solution 2: $Q = 3$ (82 points)

Using a similar idea to the previous solution, if we know which column the box is in, we can easily find its exact position by instructing the robot to go to the correct column, then moving down many times such that it gets stuck eventually.

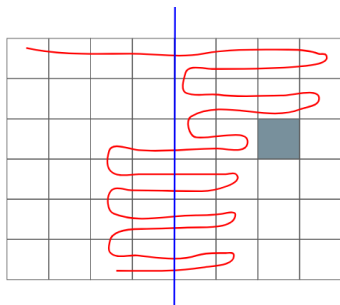
Therefore, we may consider focusing on finding the correct column of the box using 2 queries, then using 1 final query to find the position of the box. That adds up to 3 queries. Now, the question is how we can do it.

When encountering similar types of grid interactive problems, some common strategies contestants may consider include constructing some well-defined patterns (like spirals), or dividing the grid into parts, considering parities, randomized methods, etc. In fact, the first two methods are helpful with solving this task using $Q = 3$ queries!

Let's consider the following pattern: moving back and forth along each row (like in the figure below). When the robot encounters the box, we know that it gets stuck. As a result, the robot will be displaced to the left, revealing which column the box is in.



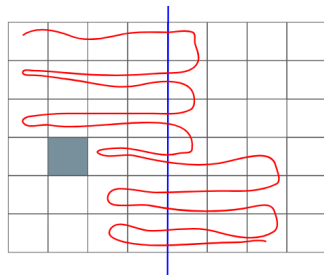
While this method seems promising, there is a small issue: the robot will run into the left wall and the information about the column will be lost. So instead, let's divide the grid into two halves and run this solution on both halves! The first query will look like this:



When the robot enters the second half of the grid, let's say it moves x units back and forth along each row. If the robot finally lands on $(H - 1, c)$ then it

must have got stuck somewhere at column $c + x$, so the box would be at column $c + x + 1$.

Of course, the above solution assumes that the box is in the right half of the grid. If the box had been in the left half, we would need one more similar query to find the correct column:



There are also some special cases, like if the box is in the first row. But if you are careful enough, this will let you solve the problem in only three queries.

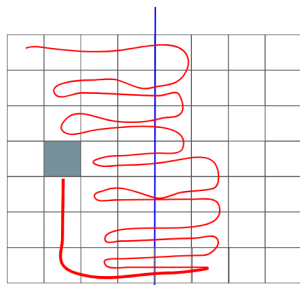
Solution 3: $Q = 2$ (100 points)

To get the number of queries down to 2, we will modify the $Q = 3$ solution.

The main idea is that we do not always need the final query to get the position given the correct column. Sometimes we can incorporate that into the query that finds the column.

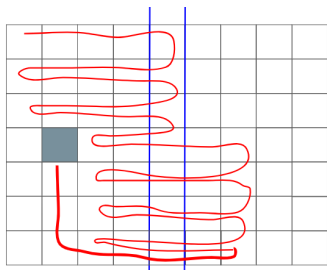
Let's start by making the same first query as in the $Q = 3$ solution. If the box was in the right half of the grid, we would find the column of the box using one query and find the row as well using another query. This takes 2 queries, so there is no issue.

The issue arises when the box was in the left half, we need to find it in only one more query. This is a bit tricky to do, and it is convenient to divide it up in two cases depending on whether W is even or odd. For even W , we can do this:



In other words, we find the column similar to how it was done in the previous solution, and then we also move back to the column and get the position of the box. This is possible thanks to the fact that we know that the box is not in the right half, so we can move freely there.

When W is odd, then this second query can look like this instead. Note that there are some special cases to be handled, such as when the box is in the middle column.



With careful handling, one may solve the problem using $Q = 2$ queries, which scores 100 points.

Aftermath

There are also many other ways to get full score on this problem. If you ask someone who solved it, then you will quite likely hear something different.

Can you prove that it is not possible to solve the problem with $Q = 1$?

Bikes vs Cars

EGOI 2023

Problem author: Nils Gustafsson.

Solution:

We can start by writing down what the numbers $C_{i,j}$ and $B_{i,j}$ mean in a more concise way:

$$C_{i,j} = \max_p(\min_b(W - b))$$

$$B_{i,j} = \max_p(\min_b(b))$$

Here, the **max** is taken over all paths from i to j , and the **min** is over all bike lane widths of edges on the path.

Note that if we let $A_{i,j} = W - C_{i,j}$, then we get rid of the parameter W , because

$$A_{i,j} = \min_p(\max_b(b))$$

So now we only focus on the bike lane width, and our goal is to construct a graph that satisfies all the **minmax**- and **maxmin**-constraints imposed by the numbers $A_{i,j}$ and $B_{i,j}$.

Subtask 1 and 2:

Here it is helpful to make the following observation:

Observation 1: The minimum edge weight in the graph is $\min(A_{i,j})$, and the maximum edge weight is $\max(B_{i,j})$.

In this subtask, this implies that if $B_{i,j} < A_{i,j}$, then it is impossible.

Otherwise, we can add edges of weight $A_{i,j}$ and $B_{i,j}$ that connect all pairs of vertices.

Subtask 3 ($N \leq 40$)

To solve the case when N is small, we need to find a construction that always works, but uses too many edges.

Observation 2: If there is a valid solution, then the following construction will also work: disregard all pairs of vertices where $A_{i,j} > B_{i,j}$. For all other pairs of vertices, add edges of weight $A_{i,j}$ and $B_{i,j}$.

Here is some motivation why this works:

First, if we have an edge between i and j of weight b , then $A_{i,j} \leq b$ and $B_{i,j} \geq b$, which means that $A_{i,j} \leq B_{i,j}$. So we can never have an edge between two vertices if $A_{i,j} > B_{i,j}$. This means that if a graph constructed as above is disconnected, then there is no solution.

Second, a path in the construction that minimizes the maximum weight between i and j will only use edges $A_{x,y}$ that we added, since they are always smaller than the $B_{x,y}$ -edges. But it could happen that the minimum maximum weight ends up smaller than $A_{i,j}$, if there is a path $i, a_1, a_2, \dots, j$ such that

$$\max(A_{i,a_1}, A_{a_1,a_2}, \dots) < A_{i,j}$$

However, if such a path existed, then it could also be used in any other construction, so in this case the answer should be NO anyway.

Third, similar arguments can be made about the $B_{i,j}$ -edges.

So all we have to do to get this subtask is to construct the graph above, and check that it is a valid solution. This check can be done by running an algorithm similar to Floyd-Warshall's, or by using minimum spanning trees.

Subtask 5 (all $B_{i,j}$ are the same)

Let B be the value of all $B_{i,j}$. Remember from subtask 1 that the maximum weight in the graph is the maximum value of $B_{i,j}$, which is B . So in this subtask, we must have that $B \geq \max(A_{i,j})$, otherwise there is no solution. But if this is the case, then we can connect all vertices with edges of weight B and then forget about the **maxmin** constraints.

After that, we only have to focus on the numbers $A_{i,j}$.

Observation 3: If we have a graph that satisfies all $A_{i,j}$ -constraints, then the minimum spanning tree of that graph will still satisfy all the $A_{i,j}$ -constraints.

This fact is a rather standard trick when it comes to minimum spanning trees. To see why it is true, assume that there exists a path from i to j such that the maximum weight is smaller than the maximum weight of the path along the minimum spanning tree. Then we could remove the largest weight edge on the

path along the tree, and replace it with an edge of smaller weight. This would create a smaller spanning tree, which is a contradiction.

So, to solve the subtask, find a minimum spanning tree of the complete graph whose edge weights are $A_{i,j}$, and check that it is a valid solution. There are several algorithms to efficiently find a minimum spanning tree, like Prim's or Kruskal's.

Full score

To get full score, we will put together the things we learned in the previous subtasks.

First, take the construction from Subtask 3. Like we saw in Subtask 5, we can take a minimum spanning tree of this graph, and it will still satisfy the **minmax**-constraints. Similarly, we can take the maximum spanning tree to satisfy the **maxmin**-constraints. Furthermore, if we take the union of these two trees, then we get a solution, if one exists.

To see why this works, note that the minimum spanning tree will only use the $A_{i,j}$ -edges added in Subtask 3, and the maximum spanning tree will only use the $B_{i,j}$ -edges. Also, a path between i and j that minimizes the maximum weight will only use the edges from the MST, like we saw in Subtask 5, and this value is exactly $A_{i,j}$ like we want (and similarly for $B_{i,j}$).

Summary of how to get 100 points: Create a graph with edges of weight $A_{i,j}$ and $B_{i,j}$ between every pair of vertices such that $A_{i,j} \leq B_{i,j}$, take the union of the minimum and maximum spanning tree, and finally check that this is a valid solution.